



Unsupervised AutoML für Anomalieerkennung

Evaluation und Modellselektion am Beispiel des Tennessee Eastman Process

Achim Duong, Jan-David Widerstein, Ferdinand Schuh
ML in Prozessindustrie • Sommersemester 2026

Inhaltsverzeichnis



1

Problemstellung und Ziel

2

Thema und Grundlagen

3

Datensatz

4

Evaluationsstrategie

5

Baseline Model

6

Vergleichsmodelle

7

Gegenüberstellung

8

Diskussion

9

Fazit

Problemstellung und Ziel



Ausgangslage

- Unternehmen betreibt chemischen Prozess
- Ziel: automatisierte Anomalieerkennung
- Keine gelabelten Fehlerdaten im Realbetrieb verfügbar



Problem

- Supervised ML scheitert aus -> Labels zu teuer
- AutoML-Frameworks fuer unsupervised AD existieren
- Aber: wie bewertet man das beste Modell ohne Ground Truth?



Ziel der Arbeit

- 1 Verschiedene AutoML- / AD-Ansätze auf TEP anwenden
- 2 Belastbare Evaluationsstrategie mit möglichst wenig Label-Abhängigkeit entwickeln
- 3 Ansätze systematisch gegenüberstellen

Fehlen eines Ground Truth ist hierbei eine bewusste Herausforderung - Labels nur für die Endevaluation genutzt



Thema und Grundlagen

AutoML

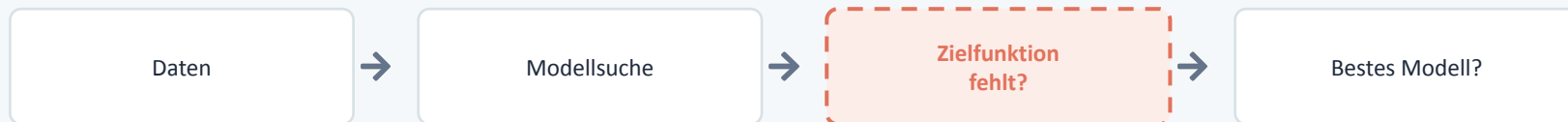


Automatisierte Auswahl und Konfiguration von ML-Modellen und Pipelines (Model Selection, Hyperparameter-Optimierung) — statt manuellem Trial-and-Error.

Klassisches AutoML (supervised)



AutoML für unsupervised Anomalieerkennung



Ohne klare Zielfunktion muss AutoML für unsupervised AD „blind“ gut funktionieren



Anomalieerkennung



Identifikation von Datenpunkten oder Mustern, die signifikant vom „normalen“ Verhalten abweichen.

Supervised

Viele gelabelte Daten

Semi-supervised

Sowohl Gutdaten als auch Fehlerzustände sind vorhanden

Unsupervised

nur Gutzustand-Daten vorhanden, keine Fehlerinfo überhaupt nötig

FOKUS DIESER ARBEIT



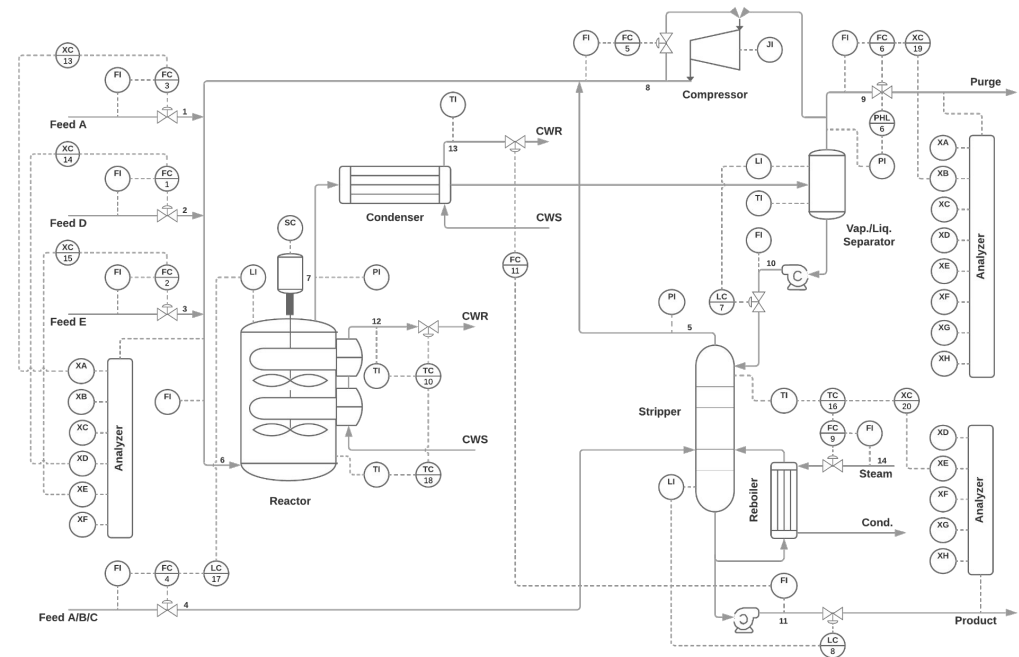
Genau dieses Fehlen von Labels macht nicht nur die Modellierung, sondern auch die spätere Bewertung der Modelle schwierig.

Was ist der Tennessee Eastman Process

- Simulierter industrieller Chemie Prozess
(leicht durch Kaggle manipuliert)
- Benchmark für Fehlererkennung & Prozessüberwachung
- Realistische, komplexe Dynamik mit nichtlinearen Verhalten

TENNESSEE EASTMAN PROCESS FLOWSHEET

Gilberto Xavier | January 8, 2018



Reference: E. L. Russell, L. H. Chiang, and R. D. Braatz, *Data-driven Methods for Fault Detection and Diagnosis in Chemical Processes*. London: Springer, 2000 (p. 105)

Copyright 2018 Gilberto Xavier

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by/4.0/> or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Datensatzstruktur auf einem Blick

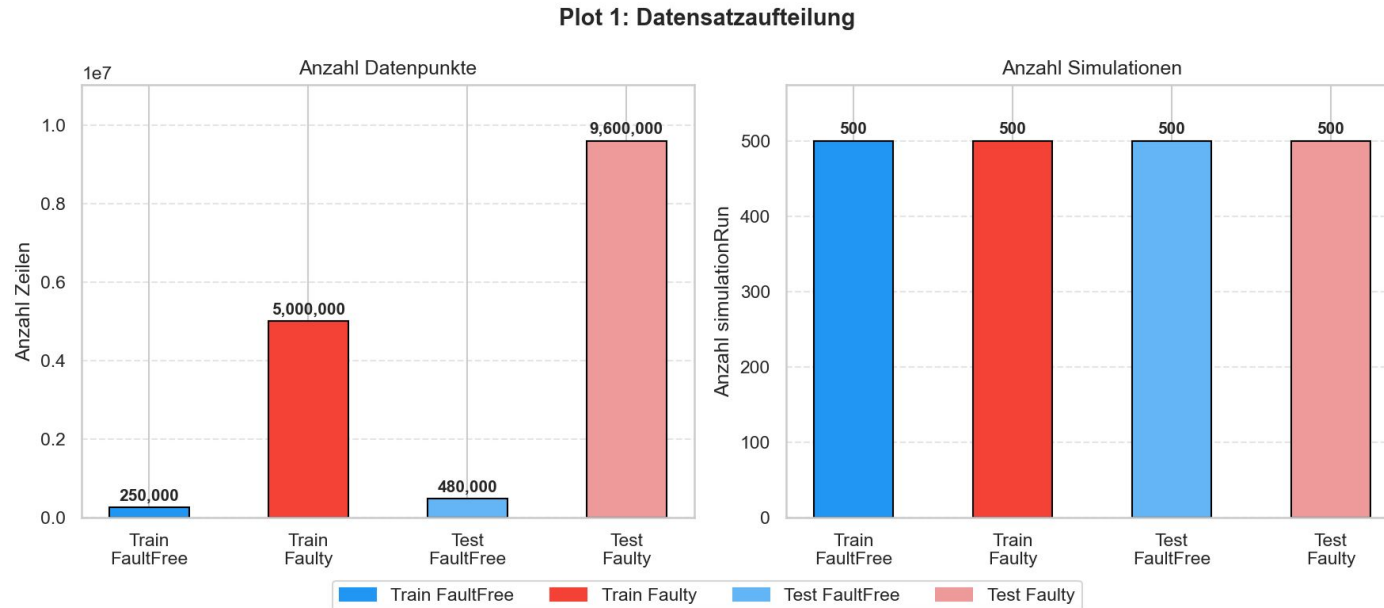
	Training	Testing
FaultFree	500 Simulationen	160 Simulationen
Faulty	480 Simulationen	840 Simulationen
Fehlertypen	20 Typen	21 Typen
Zeitschritte	500 Steps / Sim	960 Steps / Sim
Prozessvariablen	41 Mess + 11 Stellgrößen	

Die 55 Variablen - Was wird gemessen?

Kategorie	Variablen	Einheit	Anzahl
Volumenströme	xmeas_1, 4, 5, 6, 10	kscmh	5
Massenströme	xmeas_2, 3, 17, 19	kg/h	4
Temperaturen	xmeas_9, 11, 18, 21, 22	°C	5
Drücke	xmeas_7, 13, 16	kPa	3
Füllstände	xmeas_8, 12, 15	%	3
Volumenfluss	xmeas_14	m³/h	1
Leistung	xmeas_20	kW	1
Zusammensetzungen	xmeas_23–41	mol%	19

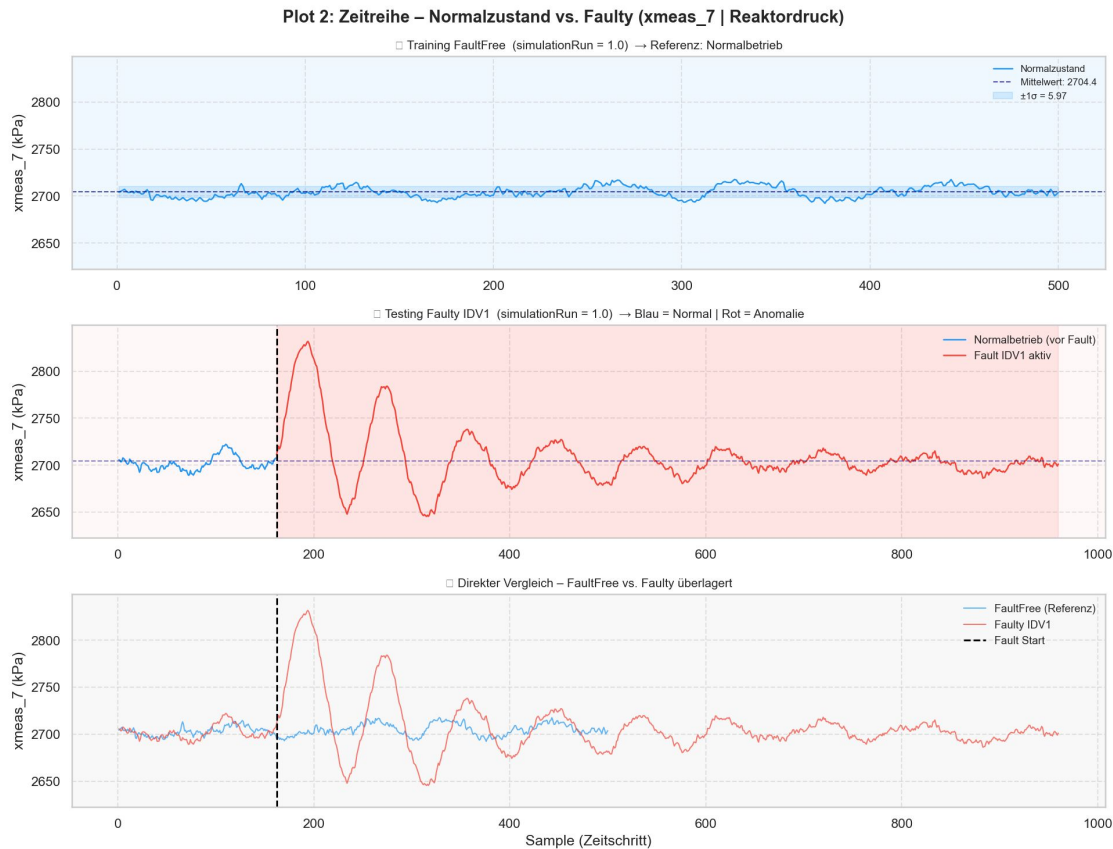
Kategorie	Variablen	Einheit	Anzahl
Ventilpositionen	xmv_1 – xmv_11	% (0–100)	11

Wie sehen die Daten aus? (I)



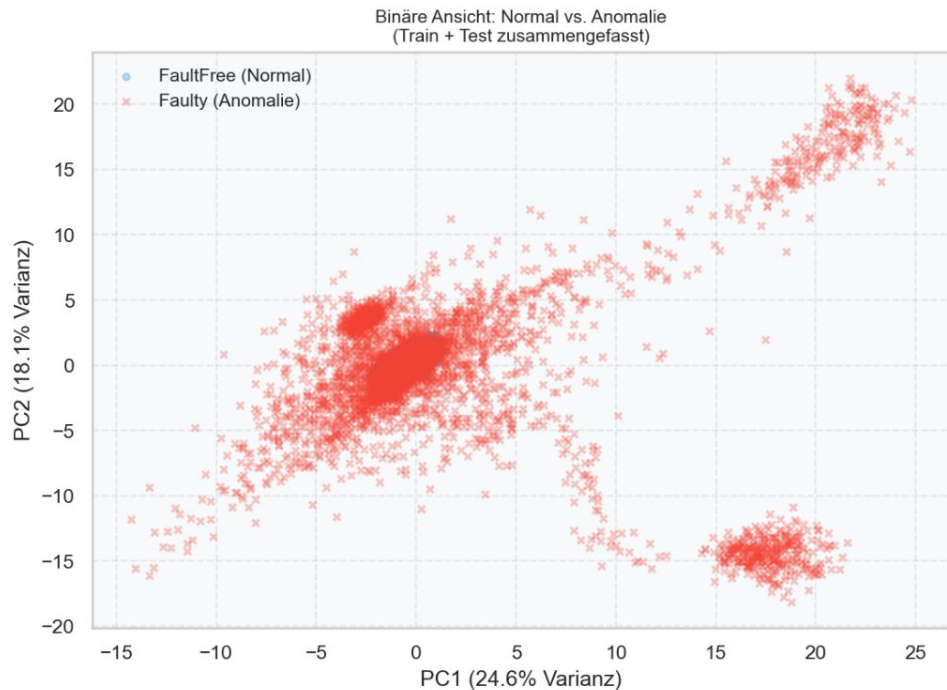
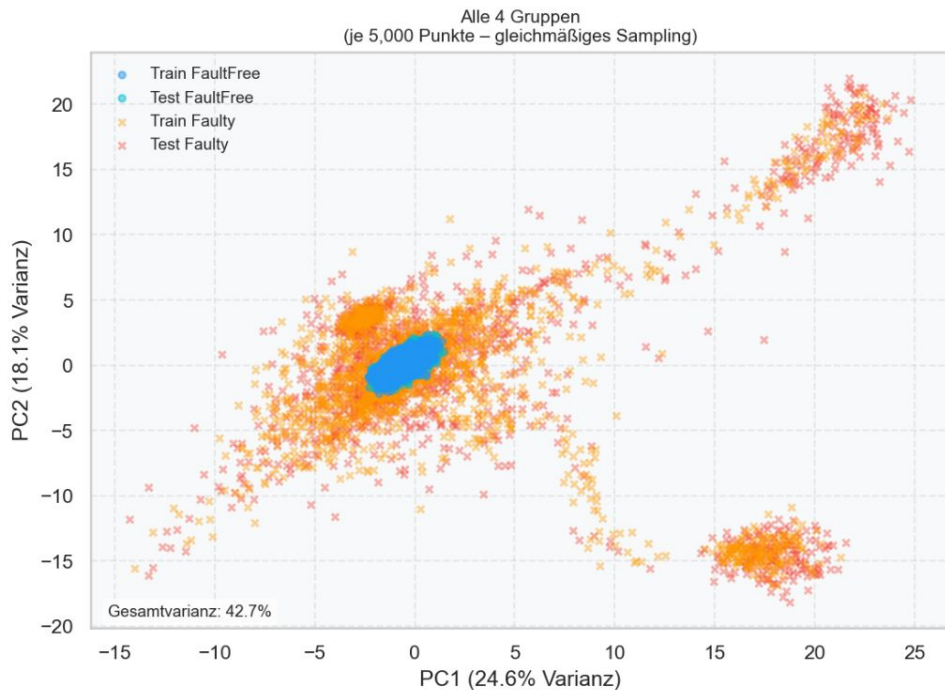
Wie sehen die Daten aus? (II)

Die ersten 160 Zeitschritte bilden den Normalzustand in Faulty



Wie sehen die Daten aus? (III)

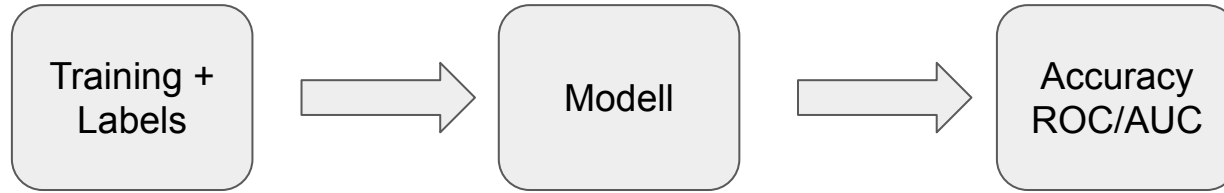
Plot 3: PCA – 2D Projektion aller 4 Dateien



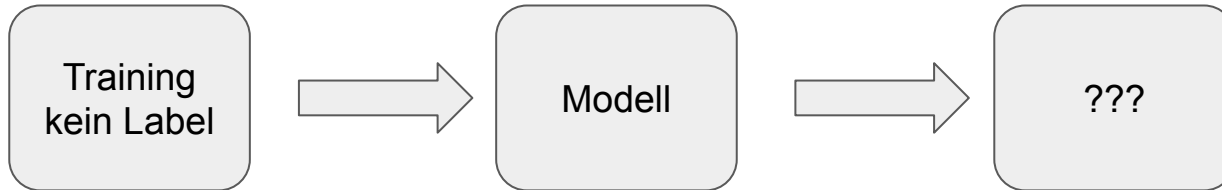
Das Evaluationsproblem

Wie bewertet man einen Anomalie Detektor ohne Labels?

- Klassisches supervised Learning:



- Unser Fall - Anomalieerkennung:



➡ Standard Metriken versagen!

PyOD

Python Library als Industriestandard für “Outlier-Detection”

Was ist pyOD?

- Stellt ein einheitliches BaseDetector-Interface für diverse Modelle zur Verfügung
- Minimale API - Entwickler müssen Algorithmen nicht selbst implementieren
- Open-Source: <https://github.com/yzhao062/pyod>

Code Beispiel:

```
docs > jd > pyod_beispiel.py > ...
1  from pyod.models.hdbscan import HDBSCAN
2  from pyod.models.knn import KNN
3  from pyod.models.pca import PCA
4  from pyod.models.iforest import IForest
5
6  import numpy as np
7
8  rng = np.random.default_rng(42)
9  X_train = rng.normal(size=(500, 8))
10 X_test = rng.normal(size=(100, 8))
11
12 for Detector in [PCA, KNN, IForest, HDBSCAN]:
13     scores = Detector().fit(X_train).decision_function(X_test)
14     print(f"{Detector.__name__:8s} max-Score = {scores.max():.2f}")
15
```

PORTS PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
• ~ /Documents/automl-ad-abgabe main +3 !
> uv run docs/jd/pyod_beispiel.py
```

PCA	max-Score = 300.63
KNN	max-Score = 3.21
IForest	max-Score = 0.05
HDBSCAN	max-Score = 1.00

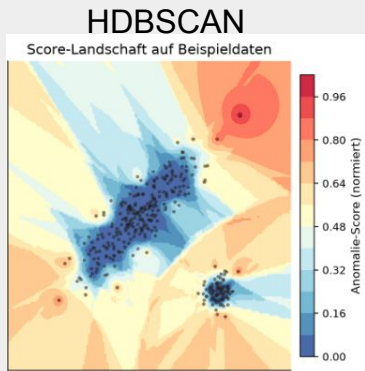
Baseline Modelle

Welche Models haben wir als Baseline verwendet?

-> Dichte Clustering

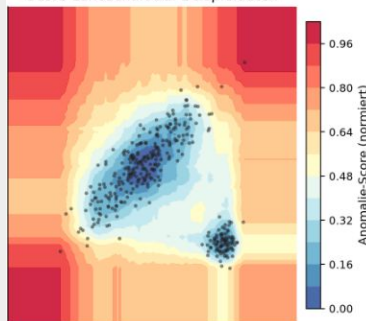
-> Wer in dünn besiedelten Regionen liegt, ist Ausreißer

Output: Dichte



Isolation Forest

Score-Landschaft auf Beispieldaten



-> Zufällige Schnitte isolieren Anomalien

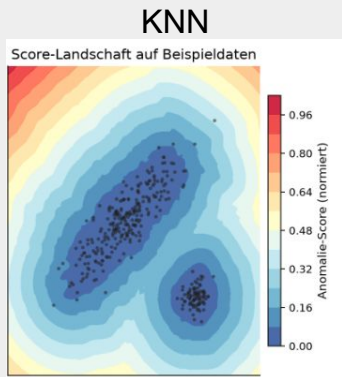
-> kurzer Pfad zum isolierten Punkt = anormal

Output: Pfadlänge

-> Abstands-basiertes Clustering

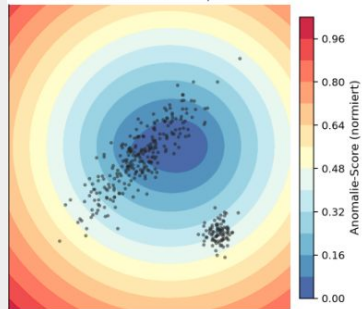
-> Wer weit entfernt ist von K-Nachbarn, ist anormal

Output: K-Distanz



PCA

Score-Landschaft auf Beispieldaten



-> Abstand Originalpunkt und seiner Projektion
-> Wenn dieser Abstand sehr groß ist, handelt es sich um eine Anomalie

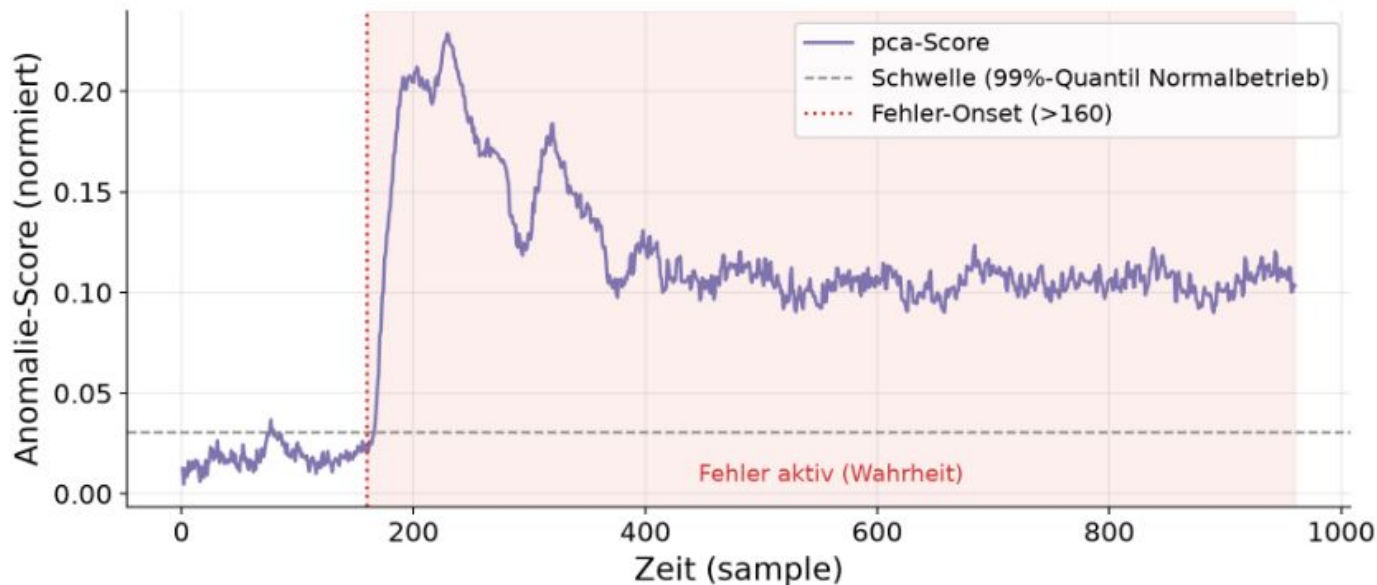
Output:
Rekonstruktionsfehler (Abstand)

Anomalie-Score im Zeitverlauf

Ein Model im Betrieb

Anlage läuft → Störung → Alarm

Nach dem Fehler-Onset bleibt der Score über der Normalbetriebs-Schwelle — das Prozess-Monitoring-Bild.

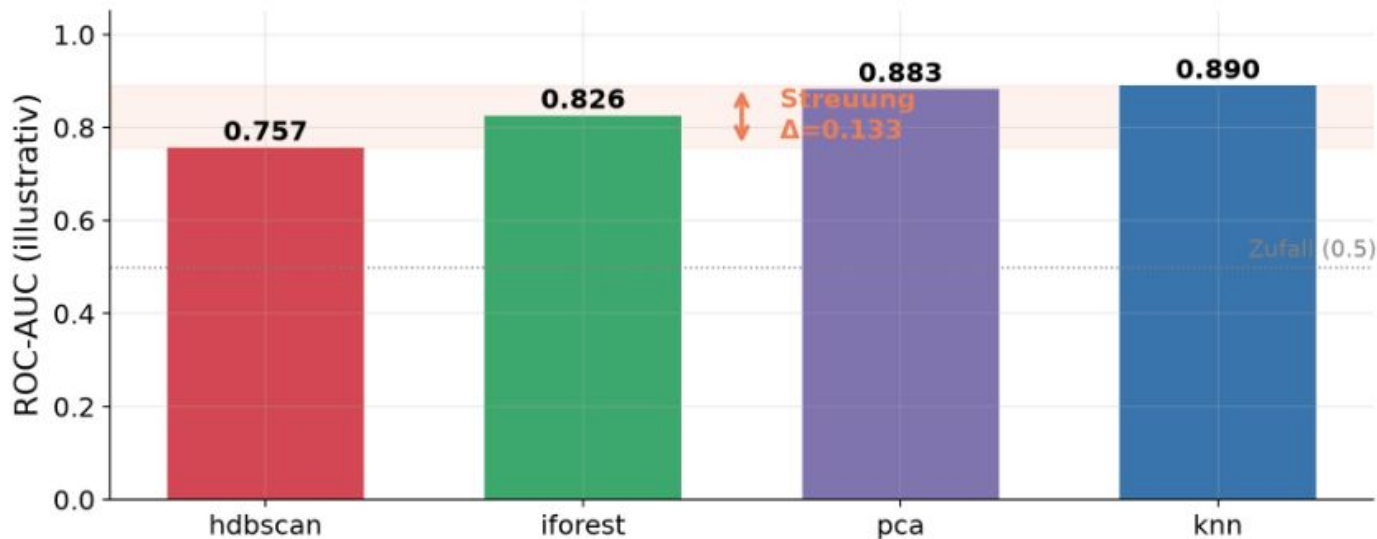


Model Selection

Einfach intuitiv ein Model aussuchen

Ohne Labels: welchen Detektor nehmen?

Die 4 PyOD-Detektoren streuen stark — blind einen zu wählen ist Glückssache.



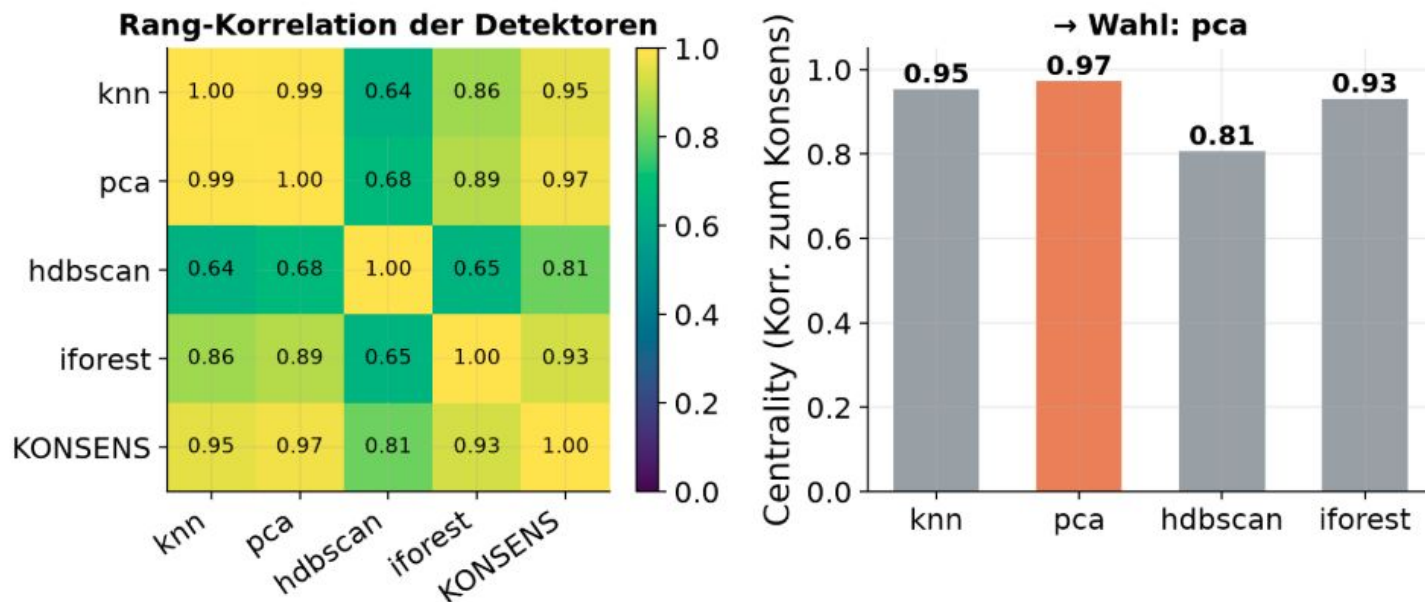
△ ROC/AUC nur zur Illustration — im echten unüberwachten Betrieb NICHT verfügbar.

Model Selection

Verschiedene Modelle vergleichbar machen - Agreement/Konsens

Konsens · Modus A: das zentralste Modell

Wähle den Detektor, der dem Rang-Konsens am nächsten ist. Schwarm-Einigkeit: agreement = 0.78.

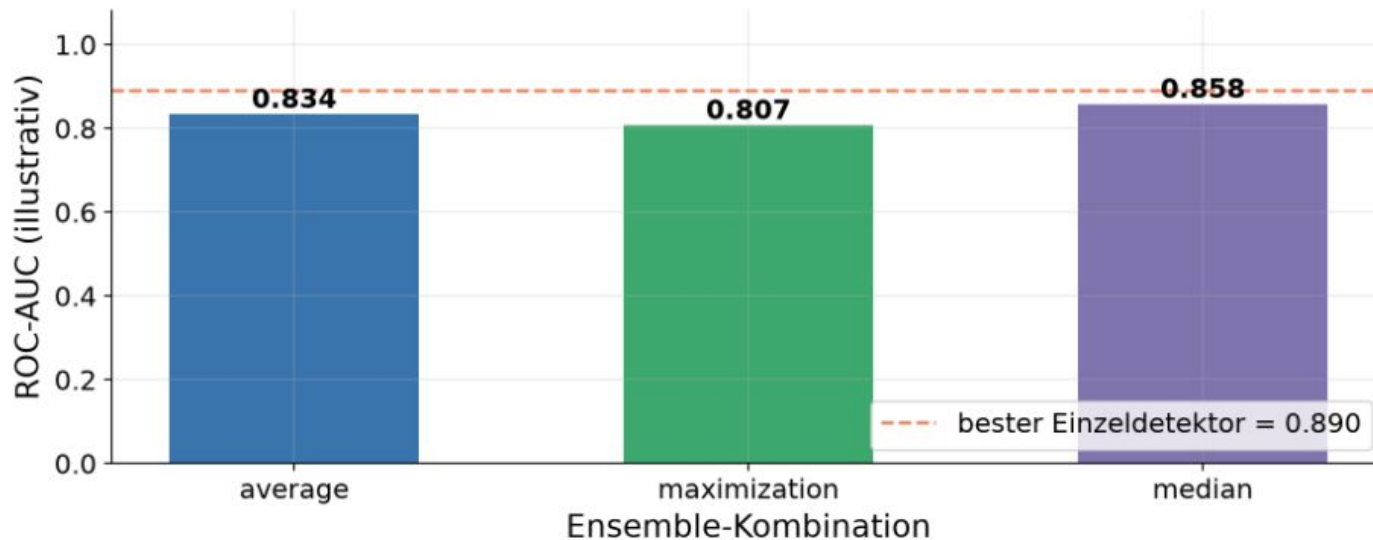


Model Selection

Verschiedene Modelle vergleichbar machen - Ensemble

Konsens · Modus B: das Ensemble ist die Vorhersage

Kein Detektor gewinnt — der kombinierte Score entscheidet, robust und unabhängig von der Einzelwahl.



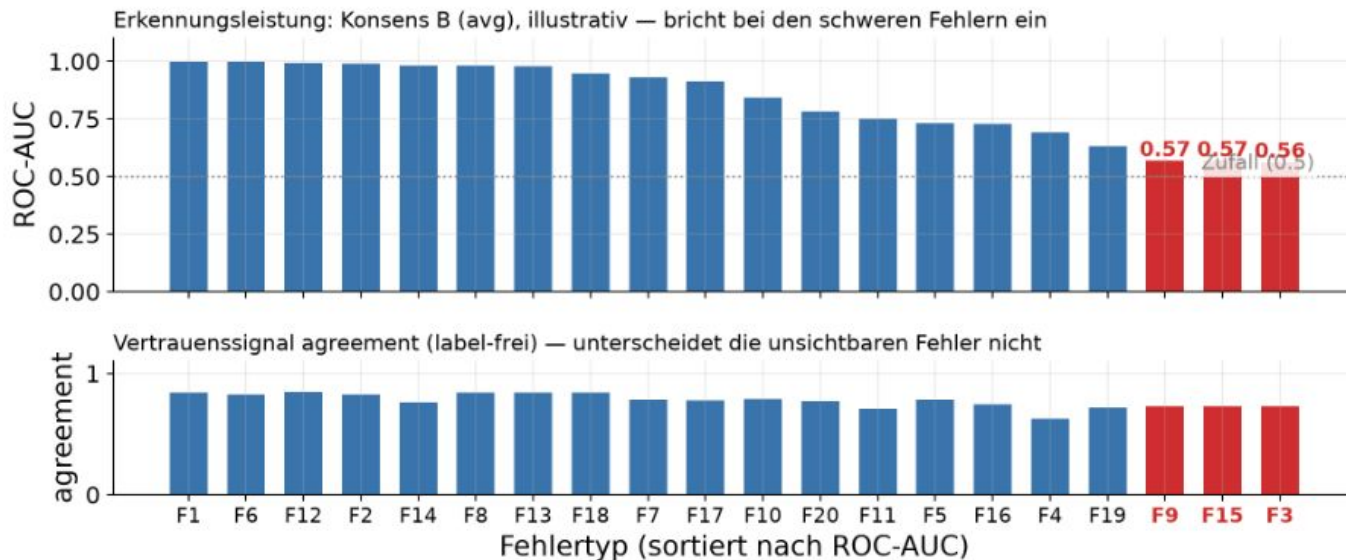
△ ROC/AUC nur zur Illustration — im echten unüberwachten Betrieb NICHT verfügbar.

Model Selection

Agreement/Konsens != Korrektheit

Differenzierung: nicht jeder Fehler ist detektierbar

Hohe Einigkeit schützt nicht vor Blindheit: das label-freie Signal zeigt die schweren Fehler nicht an (Einigkeit $\neq$ Richtigkeit).



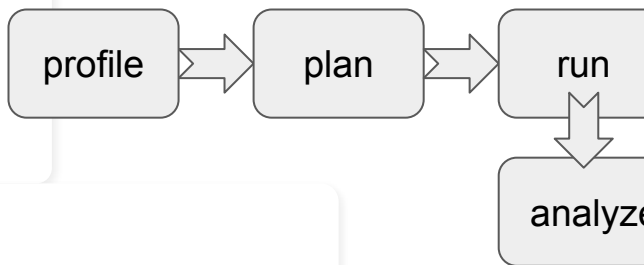
△ ROC/AUC nur zur Illustration — im echten unüberwachten Betrieb NICHT verfügbar.

pyOD - ADEngine

Automatische und Benchmark gestützte Model Selektion

Was ist die ADEngine von pyOD?

- eine "Anomalie detection lifecycle engine" (vgl. https://pyod.readthedocs.io/en/latest/pyod.ad_engine.html)
- Übernimmt Data Profiling, Planung der Detection, Erstellung der Detektoren und Benchmark gestützte Auswahl aus einer Wissensbasis (60+ Detektoren)
- Möglich über reine Python-API oder als Backend für LLM Agents / MCP Server



Code Beispiel:

```
docs > jd > adengine_beispiel.py > ...
```

```
1 import numpy as np
2 from pyod.utils.ad_engine import ADEngine
3
4 X = np.random.default_rng(42).normal(size=(500, 8))
5
6 engine = ADEngine()
7 state = engine.investigate(X, data_type="tabular")
8
9 print("Gewählte Detektoren:", [r["detector_name"] for r in state.results])
10 print("Einigkeit : ", round(state.consensus["agreement"], 2))
11 print("Qualität : ", state.quality["verdict"])
12 print("Scores : ", np.round(state.consensus["scores"][:3], 2), "...")
13
```

```
~/Documents/automl-ad-abgabe [ ] main +3 !2
> uv run docs/jd/adengine_beispiel.py
Gewählte Detektoren: ['KNN', 'IForest', 'LOF']
Einigkeit : 0.94
Qualität : medium
Scores : [0.57 0.21 0.1 ] ...
```

pyOD - ADEngine

profile

profile

Steckbrief (echter Lauf)

data_type: tabular (erkannt)
n_samples: 3 000
n_features: 52
has_nan: False
dtype: float64
dim.-Klasse: medium



Datentyp erkennen: tabular, time series, text, graph, ... (steuert die Wissensbasis)



Größe & Qualität: Samples × Features, fehlende Werte (NaN), Datentyp



Dimensionalitätsklasse steuert die Regeln:
≤ 10 low · 11-100 medium · > 100 high

52 Features



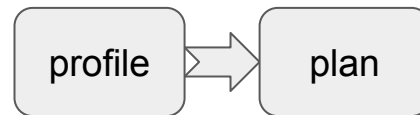
low
≤ 10

medium
11-100

high
> 100

pyOD - ADEngine

plan



Regel für dieses Profil: „general tabular → robuste Allrounder aus den ADBench-Top-5“

PRIMÄR
IForest
Rang #3 · Konfidenz 0.85

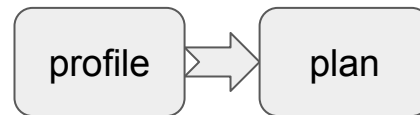
Alternative 1
ECOD
Rang #5 · Konfidenz 0.80

Alternative 2
KNN
Rang #4 · Konfidenz 0.75

- Nachschlagen statt Trainieren: Ränge stammen aus ADBench (57 fremde Datensätze) — kein Fit auf TEP.
- Der Plan enthält je Detektor: Begründung, Konfidenz und Hyperparameter (contamination = 0.1, Default).
- priority steuert die Regeln (speed | balanced | accuracy) — hier: balanced.
- Optional wählt ein LLM statt der Regeln (plan_detection(llm_client=...) → Folie LLM-Routing).

pyOD - ADEngine

plan

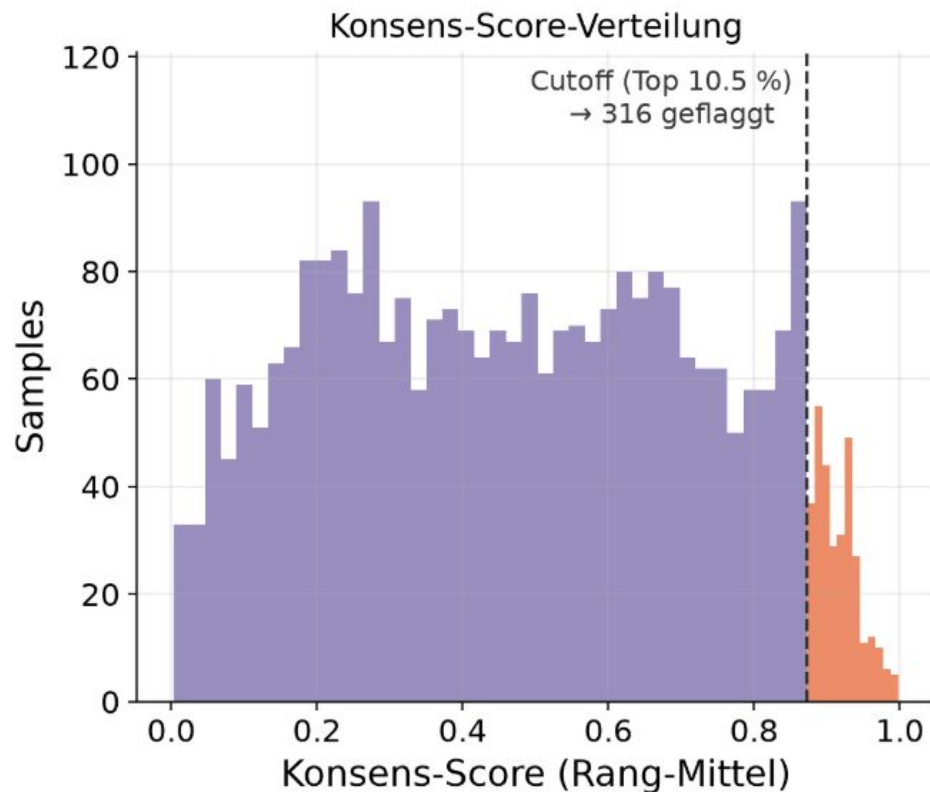


```
16 profile = engine.profile_data(X)
17 plan = engine.plan_detection(profile, priority="balanced")
18 print("Profil :", profile)
19 print("Wahl  :", plan["detector_name"], "-", plan["reason"])
20 print("Konfid.:", plan["confidence"], "· Params:", plan["params"])
21 print("Altern.:", [a["detector_name"] for a in plan["alternatives"]])
22
23 info = engine.explain_detector(plan["detector_name"])
24
25 print("Stärke  :", info["strengths"][0])
26 print("Schwäche:", info["weaknesses"][0])
27 print("Meiden  :", info["avoid_when"])
```

```
~/Documents/automl-ad-abgabe [ ] main +3 !29 ?19 .....
> uv run docs/jd/adengine_plan_beispiel.py
Profil : {'data_type': 'tabular', 'n_samples': 3000, 'n_features': 52, 'has_nan': False, 'dimensionality_class': 'medium'}
Wahl  : IForest – General tabular: robust all-rounders from ADBench top-5
Konfid.: 0.85 · Params: {'contamination': 0.1}
Altern.: ['ECOD', 'KNN']
Stärke  : Excellent overall benchmark performance
Schwäche: May struggle with local anomalies in dense regions
Meiden  : Anomalies are local density deviations or features are strongly correlated
```

pyOD - ADEngine

run



1) Jeder Detektor fittet & scort auf X:

IForest	0.11 s	flaggt Top-10 % = 300 Stk.
ECOD	0.26 s	flaggt Top-10 % = 300 Stk.
KNN	0.03 s	flaggt Top-10 % = 300 Stk.

2) Rang-Konsens (wie Modus A):

Scores je Detektor → Ränge → Mittel

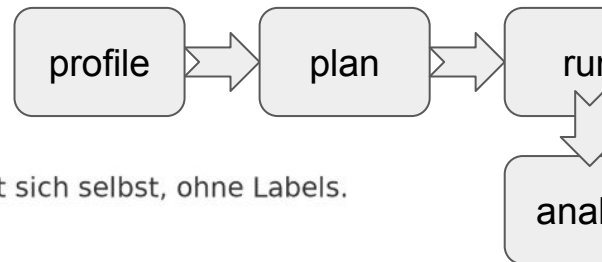
Konsens flaggt 316 / 3 000 (10.5 %)

agreement = 0.79 (Schwarm-Einigkeit)

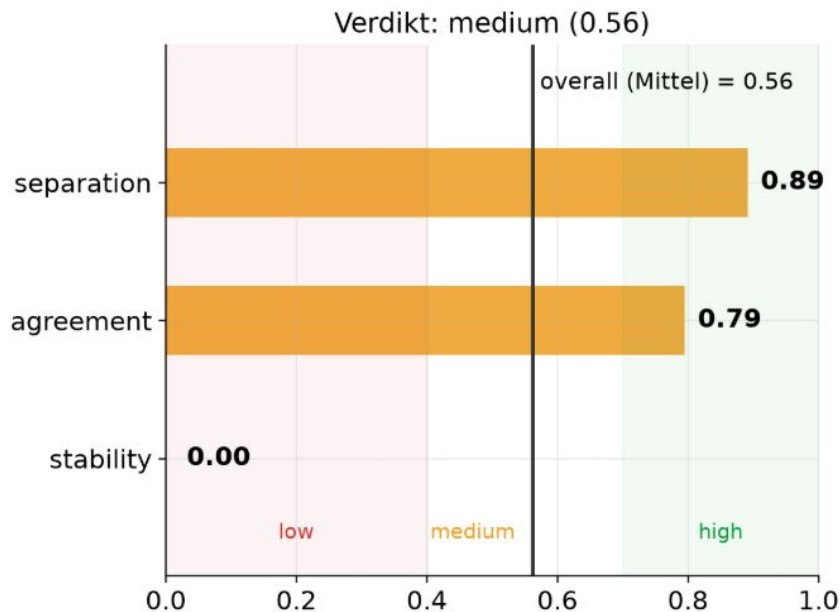
Streitfälle: 432 Samples uneinig

pyOD - ADEngine

analyze



verdict = Mittel aus separation · agreement · stability — die Engine benotet sich selbst, ohne Labels.



separation 0.89

Geflaggte heben sich klar vom Rest ab — aber gemessen an den EIGENEN Labels (deskriptiv).

agreement 0.79

Die 3 Detektoren ranken sehr ähnlich (mittlere paarweise Spearman-Korrelation).

stability 0.00

Kein Score-Sprung am Cutoff: die 10 %-Grenze ist wackelig → contamination passt nicht (TEP > 10 %).

best_detector

label-frei gewählt: ECOD — die Selbst-Bewertung kann sich irren (Einigkeit ≠ Richtigkeit).

Schwellen: overall ≥ 0.7 high · ≥ 0.4 medium · sonst low.

pyOD - ADEngine

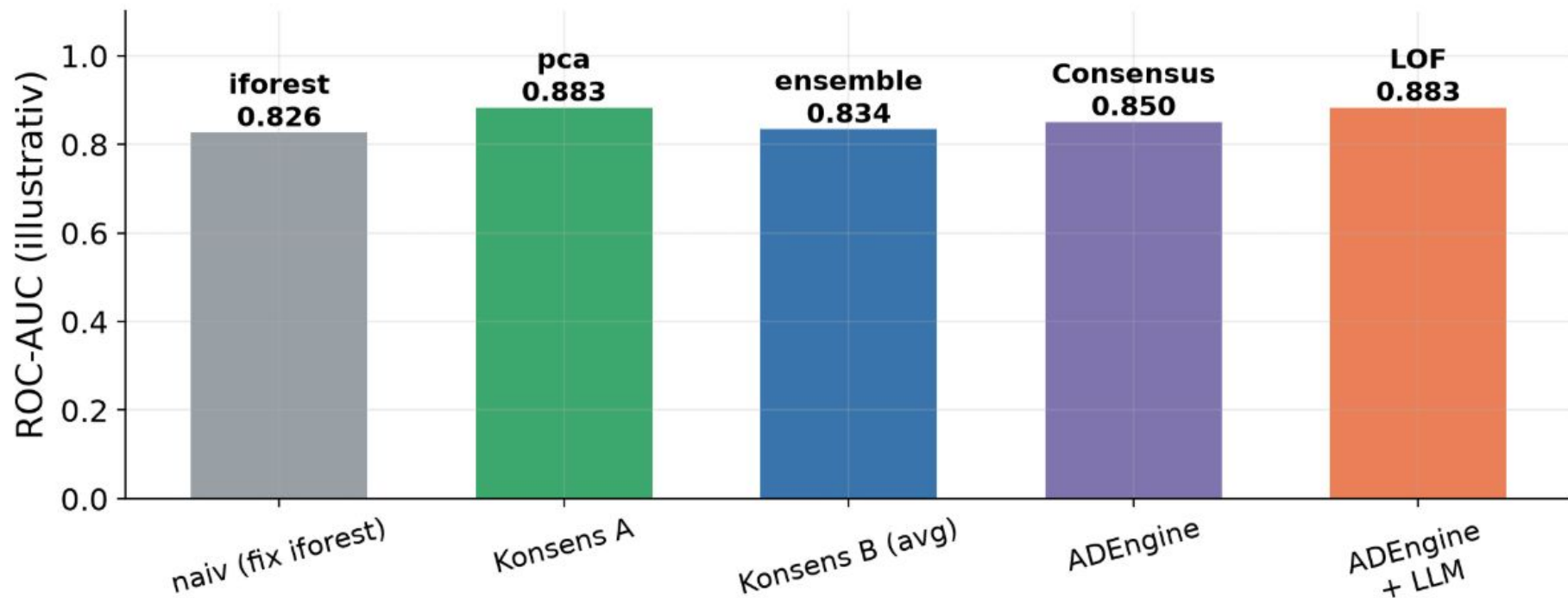
LLM Adapter



LLM wählte: LOF (ROC-AUC 0.883)

- ① Sicher: kann nicht ausbrechen / nichts Ungültiges (Form + KB-Validierung + Regel-Fallback).
- ② Nicht-deterministisch: Wahl schwankt je Lauf → wir cachen eine gute (make cache).

pyOD - Fazit

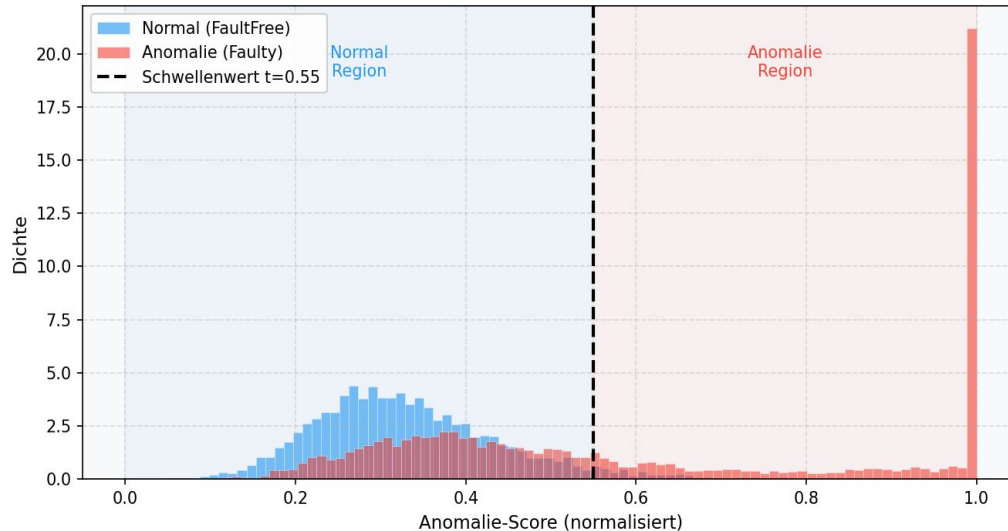


Excess Mass & Mass Volume

Evaluation ohne Labels

Idee: Gute Modelle komprimieren den Normalzustand in eine kleine Region

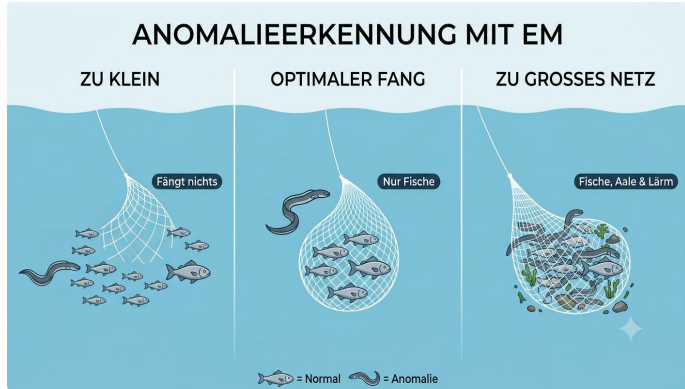
Option A - Score-Verteilung: Normal vs. Anomalie



1. Kein Label nötig
-> nur Anomalie-Scores des Modells nötig
2. Zwei Fragen:
 - EM: Treffe ich viele Anomalien mit kleiner Region?
 - Wie klein ist die Region für Anomalien
3. Modellunabhängig
-> Isolation Forest, KNN, PCA, HDBSCAN vergleichbar

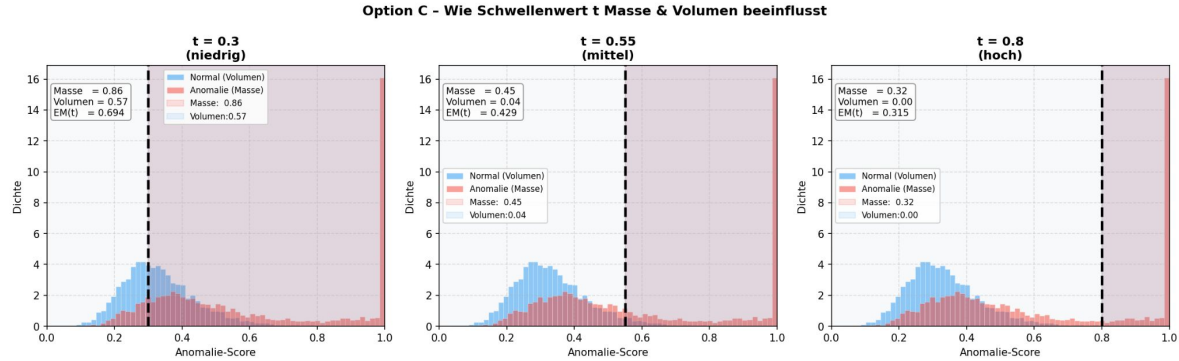
Excess Mass - “Treffe ich viel mit wenig?”

Stell dir vor du wirfst ein Netz ins Meer:



Gutes Modell = Viele Fische, wenige Aale

Anwendung auf unseren TEP-Datensatz:



$$EM(t) = \text{Masse} - t \times \text{Volumen}$$

Mass Volume - “Wie kompakt ist meine Region?”

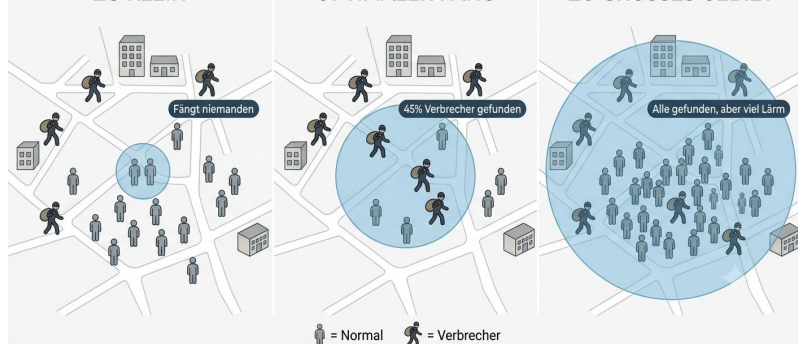
SUCHGEBIETSANALOGIE FÜR MASS VOLUME (MV)

Kernfrage: “Wie groß muss mein Suchgebiet sein, um 45% der Verbrecher zu finden?”

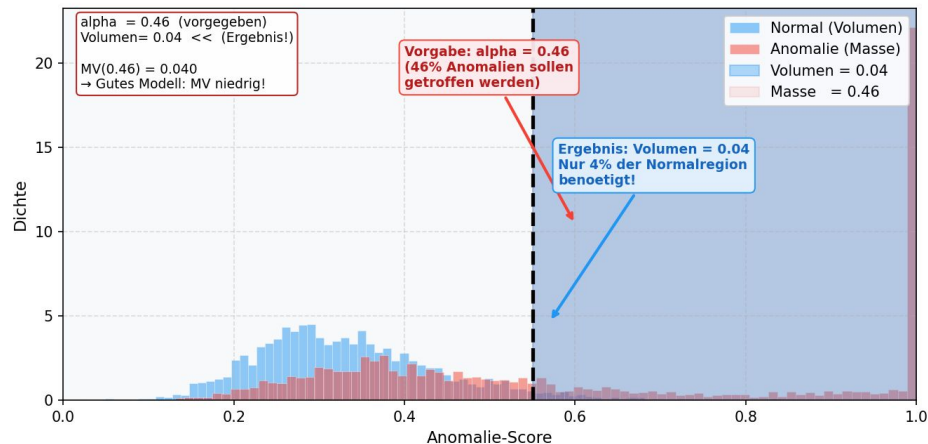
ZU KLEIN

OPTIMALER FANG

ZU GROSSES GEBIET



**MV - Ich wähle $\alpha=0.46$ (46% Anomalien):
Wie groß ist meine Region dafür?**

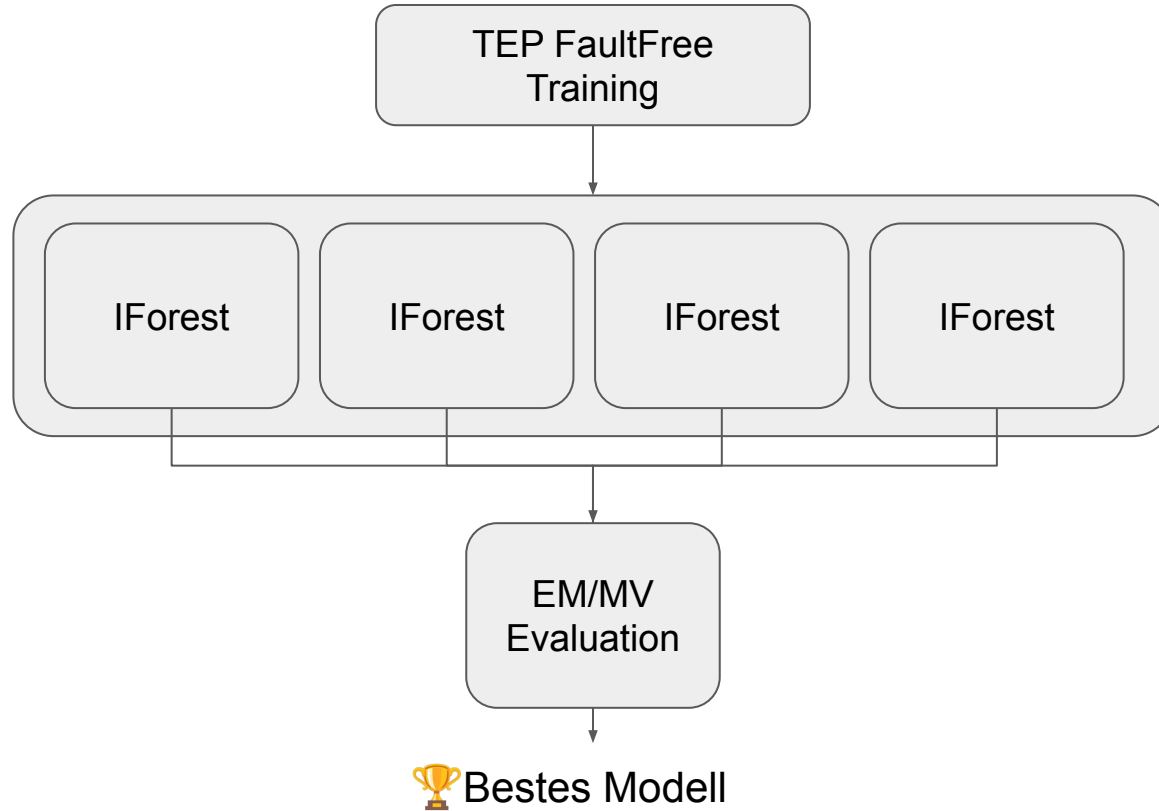


MV(α) = Wie groß ist das Suchgebiet für $\alpha\%$ Anomalien?

-> Kleines Gebiet = Guter Detektor ✓

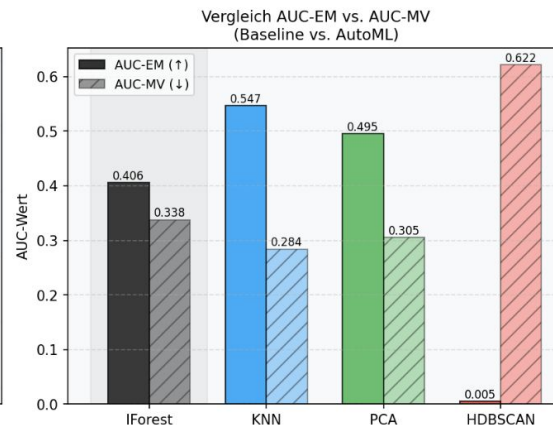
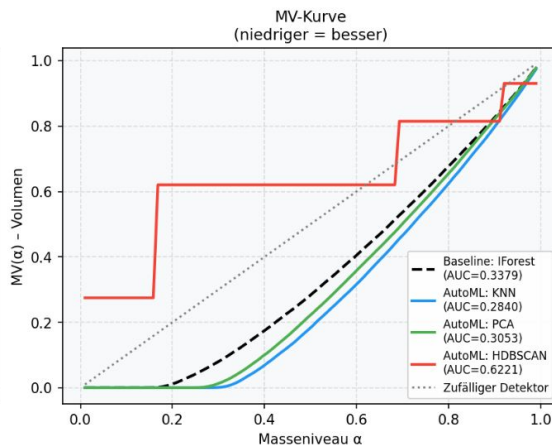
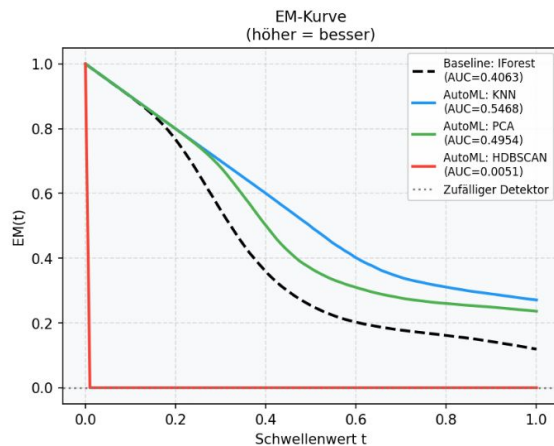
-> Großes Gebiet = Schlechter Detektor ✗

AutoML Pipeline mit EM/MV als Entscheider



EM/MV Ergebnisse

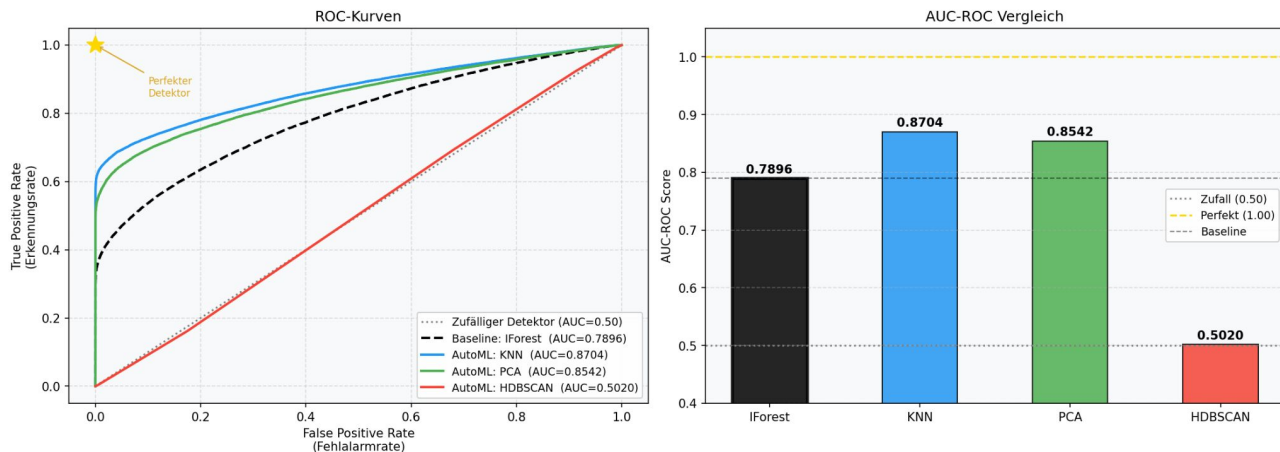
EM/MV Evaluation: Baseline vs. AutoML Modelle



Modell	AUC-EM	Rang	AUC-MV	Rang
KNN	0.5468	1	0.2840	1
PCA	0.4954	2	0.3053	2
IForest	0.4063	3	0.3379	3
HDBSCAN	0.0051	4	0.6221	4

ROC/AUC Vergleich

ROC/AUC Kurven - Baseline vs. AutoML Modelle
(möglich durch gelabelte TEP-Daten)



Modell	AUC-EM	Rang	AUC-ROC	Übereinstimmung
KNN	0.5468	1	0.8704	
PCA	0.4954	2	0.8542	
IForest	0.4063	3	0.7896	
HDBSCAN	0.0051	4	0.5020	

MetaOD



Was ist MetaOD?

- Meta-Learning-Verfahren zur unsupervised Anomalieerkennung
- Vorhersage anhand von Datensatz-Charakteristiken
- Benötigt kein Ground-Truth des Zieldatensatzes

Kernidee

- Lernt die Beziehung zwischen Daten-Charakteristiken und bekannter Modellperformance auf vielen gelabelten Benchmark-Datensätzen
- => Nutzt dieses gelernte Muster, um passende Modelle für neue, ungelabelte Datensätze vorzuschlagen



Technischer Ablauf: Training & Inferenz

Training (offline, einmalig)



Performance-Matrix aus dem Training ist die Grundlage für das Matching in der Inferenz

Inferenz (online, für TEP)



Meta-Features, z. B.: Dimensionalität, Skalierung, statistische Momente, Korrelationsstruktur der Daten.



Ergebnisse auf TEP



Vorläufig: Subsample (10 Läufe/Fehlertyp, 1 Seed, Top-8 Empfehlungen). Tendenzen belastbar, exakte Werte werden mit hochskaliertem Testlauf bestätigt.

Rang	Modell	ROC-AUC	Laufzeit
1	ABOD 5	0.872	42 s
2	kNN (5, median)	0.891	6 s
3	ABOD 25	0.891	673 s
4	ABOD 10	0.886	110 s
5	ABOD 20	0.891	391 s
6	LOF (100, manhattan)	0.876	36 s
7	ABOD 3	0.837	17 s
8	ABOD 15	0.889	223 s

kNN hervorgehoben: nahezu identisches ROC-AUC bei einem Bruchteil der Laufzeit.

Wichtigste Erkenntnisse

- MetaODs Top-Empfehlung (ROC-AUC 0.872) liegt nah, aber unter dem Bestwert (0.891) — gut, nicht optimal
- Alle Top-Empfehlungen liegen eng beieinander (0.837–0.891)



ABOD — Angle-Based Outlier Detection



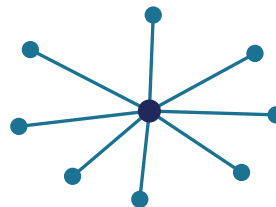
Kernidee

- Misst die Varianz der Winkel zwischen einem Punkt und allen Paaren anderer Punkte in seiner Umgebung
- Normale Punkte (dichte Region): Nachbarn liegen in vielen Richtungen → hohe Winkel-Varianz
- Ausreißer (isoliert/am Rand): Nachbarn bündeln sich in einer Richtung → geringe Winkel-Varianz
- Anomalie-Score = inverse Winkel-Varianz
- ABOD n: $n = k$ nächste Nachbarn



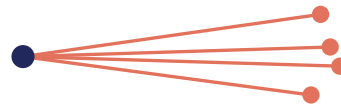
Stärken & Schwächen

- + Robust in hochdimensionalen Räumen (Winkel stabiler als Distanzen)
- Rechenintensiv: skaliert schlecht (ABOD 25 ≈ 11 Min. Laufzeit)



Normaler Punkt

hohe Winkel-Varianz



Ausreißer

geringe Winkel-Varianz



Vergleich MetaOD vs. Konsens

Modell	Beschreibung	ROC-AUC	Konsens-Nähe
ABOD 5	Höchster Rang	0.872	0.927
kNN (5, median)	Model mit höchstem ROC-AUC	0.891	0.964

Spearman-Korrelation der Top-10 MetaOD Detektoren mit Konsens: **0.93**

=> Bessere Detektoren aus MetaOD liegen näher am Konsens



Vergleich MetaOD mit EM/MV

Modell	Beschreibung	ROC-AUC	EM-AUC	MV-AUC
ABOD 5	Höchster Rang	0.872	0.318	0.0197
kNN (5, median)	Model mit höchstem ROC-AUC	0.891	0.349	0.0180

Spearman-Korrelation des ROC-AUC der Top-10 MetaOD Detektoren mit EM/MV:

+ 0.32
EM-AUC

- 0.33
MV-AUC